

TP 5 - Clases Abstractas vs. Interfaces

¿Qué ventajas, en cuanto a polimorfismo, brindan las interfaces en Java?

La ventaja en cuanto al polimorfismo es la posibilidad de que se programa para una abstracción y no para una implementación.

Es decir, se depende de un concepto genérico y no de un objeto específico en particular. Diseñar para los contratos que se deben cumplir de las interfaces y delegar los detalles a las clases.

¿Por qué no siempre puedo utilizar clases abstractas para agrupar clases polimórficas?

En principio una clase solo puede heredar de otra, entonces el comportamiento se limita a lo declarado en la superclase. En cambio, las interfaces permiten que una clase pueda implementar múltiples interfaces. Además, las interfaces modelan habilidades comunes de clases que no tenga nada que ver en común, flexibilizando su comportamiento. En adición, con una interfaz nueva puedes hacer que otras clases implementen su contrato, mientras que con una clase nueva, solamente van a poder imitar su comportamiento las clases que hereden de esta, pero no así las clases ya declaradas.

En síntesis, las clases abstractas se basan en un polimorfismo rígido y jerárquico en comparación con las interfaces con su polimorfismo flexible y horizontal.

¿Qué ventajas tienen las clases abstractas sobre las interfaces?

Las clases abstractas pueden tener atributos normales, las subclases heredan automáticamente su comportamiento y sus atributos. Se puede instanciar subclases con constructores. Al ser una clase, sus métodos pueden ser privados si así lo deseamos, cosa que no sucede con las interfaces. Además la clase abstracta tiene la posibilidad de permitir patrones de diseño como el template method. Mientras que la interfaz te dice que puede hacer un objeto, la clase abstracta te dice que es el objeto

junto a su estado y la posibilidad de incorporar el comportamiento de su superclase.

¿Se puede instanciar una interface?

No se puede. La interfaz no debería contar con atributos, solo la firma de los metodos que podría realizar el objeto que la implemente. La manera en la cual se lleva a cabo cada firma depende de la clase que la implemente.

¿Por qué no es recomendable incrementar o modificar las firmas definidas en un interface?

El hecho de incrementar o modificar una interfaz implica tener que modificar cada una de las clases que implementan esa interfaz. Por modificación debería emparejar con el nombre de la firma y por agregación no permitiría compilar porque hay metodos sin implementar.

¿Por qué, en lenguajes como Smalltalk, no es necesaria la implementación de interfaces?

A diferencia de Java y su lenguaje tipado, Smalltalk cuenta con Duck Typing, lo que implica que de los objetos importan sus comportamientos y no que son en si. Smalltalk no necesita interfaces, el polimorfismo está implícito en los mensajes que envían y reciben los objetos únicamente.